

Feature Engineering Rationale for Behavioral Intrusion Detection in SDN-Based IoT Networks

*A Systematic Justification of the Selected Feature Set and the
Rejection of Alternative Approaches*

Graduation Project :
SDN-Based IoT Intrusion Prevention System
2025 / 2026

Abstract

— This document provides a systematic justification for the ten feature groups selected for the behavioral network traffic dataset generated by the SDN-based IoT Intrusion Prevention System. For each feature group, the document articulates the specific attack mechanism that the features are designed to expose, explains the mathematical and network-theoretic basis for the chosen metrics, and explicitly addresses why simpler or more commonly used alternatives were insufficient for the threat model under consideration. The discussion is grounded in the specific architectural constraints of the system—namely, an OpenFlow 1.0 software-defined network mirroring all data-plane traffic to a Ryu controller—and addresses the unique challenges introduced by the Mininet-WiFi emulation environment, including broadcast amplification artifacts and the absence of physical propagation delay. The paper concludes that the chosen feature set provides multi-scale behavioral visibility spanning flow-level mechanics, device-level historical profiling, network-level entropy, and protocol-specific behavioral fingerprints, and that this combination is both necessary and sufficient for training a deep learning model capable of detecting not only known attack patterns but behavioral anomalies consistent with previously unseen attack variants.

Index Terms—*Software-Defined Networking, Intrusion Detection Systems, Feature Engineering, Behavioral Profiling, Shannon Entropy, Network Anomaly Detection, Deep Learning, IoT Security.*

I. Introduction

The construction of a labeled network traffic dataset for machine learning is not a neutral engineering task. The choice of which packet attributes to measure, how to aggregate them across time windows, and how to represent device history constitutes a set of implicit hypotheses about what network attacks look like at the feature level. A dataset built on the wrong hypotheses will train a model that learns to recognize the specific tools used during data collection—hping3, nmap, arpspoof—rather than the underlying behavioral principles that make those tools detectable. Such a model will fail the moment an attacker uses a different tool, a different rate, or a different payload pattern.

This document presents the reasoning behind each of the ten feature groups incorporated into the enhanced dataset schema. The goal is not to enumerate the features themselves—that documentation exists in the system's codebase—but to explain why those **particular measurements** were chosen over the many plausible alternatives, and why the alternatives were rejected.

The analysis is organized as follows. Section II describes the architectural constraints that shaped feature selection. Sections III through XII each address one feature group in detail. Section XIII compares the resulting feature set against established benchmark datasets. Section XIV concludes with a synthesis of the design philosophy.

II. Architectural Constraints on Feature Design

Before motivating individual feature choices, it is necessary to establish the environmental constraints that ruled out certain otherwise attractive approaches.

A. The Controller as a Passive Observer

The system operates a Ryu SDN controller that receives a full mirror of every data-plane packet via the OpenFlow OFPP_CONTROLLER action with `max_len=0xffff`. This means the controller observes the complete packet stream—headers and payloads—for every flow. This is an unusually privileged vantage point compared to, for example, a NetFlow-based sensor that receives pre-aggregated statistics from a hardware router. **The consequence for feature design is that per-packet granularity is available but not free: each packet triggers a Python function call, and the computational budget for feature accumulation is bounded by the packet arrival rate.**

This constraint ruled out features that require deep packet inspection of application-layer payloads, such as HTTP method distribution or DNS query content analysis. While such features have strong discriminative power, extracting them at 150,000 packets per second in a single-threaded Python process would cause the controller queue to saturate before the attack data could be recorded. The feature set was therefore restricted to transport-layer and network-layer header fields, plus aggregate statistics derivable from those fields within a constant memory budget per flow.

B. The Mininet-WiFi Amplification Problem

The data plane is emulated by Mininet-WiFi, which introduces a non-trivial measurement artifact: every packet is processed by the OpenFlow software switch and mirrored to the controller twice—once through the switch's data path and once through the TAP interface clone. **This doubles all raw packet counts and byte counts relative to what a physical sensor would observe.**

This constraint ruled out any feature whose validity depends on absolute packet counts being accurate physical measurements. Features expressed as ratios—such as the ratio of SYN packets to ACK packets, or the ratio of forward traffic to backward traffic—are immune to this amplification because the numerator and denominator are inflated by the same factor. Features expressed as entropies of frequency distributions are similarly immune because the relative frequencies are preserved under uniform scaling. Raw PPS values, however, are inflated by a factor of approximately two and must be interpreted in that context. The threshold values in the detection engine (15,000 packets per window for ICMP floods, 5,000 for SYN floods) **have been calibrated to account for this amplification.**

C. The Five-Second Window Constraint

All features are computed per five-second aggregation window per source IP. This temporal granularity is a design choice with significant implications. A shorter window—one second, for example—would capture finer temporal dynamics but would produce fewer packets per window for low-rate normal traffic, making many statistical features unreliable due to small sample sizes. A longer window—thirty seconds—would smooth out the distinction between a brief spike and a sustained attack, **compromising the consecutive-window confirmation system. Five seconds was selected as the minimum window duration** at which a flood attack produces enough packets to populate all statistical features reliably, while still being short enough to confirm a sustained attack within fifteen seconds.

This constraint ruled out session-level features that require observing the complete lifecycle of a TCP connection, since a single connection may span many five-second windows. Session completeness features are therefore computed as approximations within the window boundary rather than as exact measurements of individual connection lifecycles.

III. Timing and Inter-Arrival Rhythm Features

A. The Core Hypothesis

The fundamental hypothesis motivating this feature group is that human-generated network traffic and tool-generated attack traffic differ not only in volume but in temporal regularity. A user browsing the web generates packets at intervals shaped by human reaction time, server processing delay, and network propagation—intervals that are highly irregular and follow a heavy-tailed distribution. A flood tool running on modern hardware generates packets at intervals constrained only by the operating system's network stack and the CPU clock, producing an interval distribution that is extremely narrow and concentrated near a single value.

This hypothesis is operationalized through the coefficient of variation of inter-arrival times, defined as the ratio of the standard deviation to the mean of the inter-arrival time series. A coefficient of variation near zero indicates robotic regularity; a coefficient of variation substantially greater than zero indicates the

natural irregularity of human-initiated or application-driven traffic. **Crucially, this metric is sensitive to attack behavior regardless of the attack's volume**—a slow-rate flood at 100 PPS is just as temporally regular as a high-rate flood at 100,000 PPS, while a legitimate 100 PPS data transfer exhibits natural jitter.

B. Why Not Use PPS Alone

The most common approach in threshold-based intrusion detection systems is to trigger an alert when the packets-per-second rate from a given source exceeds a fixed threshold. This approach has a well-known failure mode: the threshold must be set high enough to avoid false positives during legitimate high-volume traffic (such as iperf throughput tests), but setting it high also means that moderate-rate attacks—those deliberately kept below the threshold—go undetected. More fundamentally, PPS is a scalar quantity that discards all temporal structure. Two flows with identical mean PPS can have radically different inter-arrival distributions: one perfectly uniform (attack tool) and one highly irregular (legitimate application). **The inter-arrival coefficient of variation captures this distinction; PPS alone cannot.**

C. Why Not Use Payload Content for Timing

An alternative approach is to infer attack timing from application-layer content—for example, by observing that all HTTP requests have identical User-Agent strings or that all DNS queries are for the same domain. While such features have high precision for known attack tools, they require deep packet inspection, **which is computationally expensive and fails entirely against encrypted traffic.** Transport-layer timing features require only the arrival timestamp of each packet, which is available at zero additional parsing cost in the controller's packet_in handler.

D. The Burst Detection Rationale

The burst_count and burst_duration_avg features address a specific attack variant not captured by the coefficient of variation: pulsed attacks that alternate between periods of very high rate and brief pauses. Such attacks are sometimes used to evade window-based rate counters that measure average PPS over the full window—the pauses bring the average below the threshold even though the peak rate during bursts is clearly malicious. Burst detection identifies periods within the window where the instantaneous rate exceeds three times the window mean, counting how many such periods occurred and how long they lasted on average. An attack conducting ten bursts of 500ms each per five-second window will have a burst_count of ten and a burst_duration_avg of 0.5, while a legitimate high-throughput flow will have a burst_count near zero.

IV. Directionality and Asymmetry Features

A. The Asymmetry Principle

Network attacks directed at a target host are, by definition, asymmetric: the attacker transmits large volumes of traffic toward the victim, and the victim either fails to respond (due to resource exhaustion) or responds minimally (with TCP RST or ICMP unreachable messages that are negligible in volume relative to the

attack traffic). This asymmetry is one of the most universal properties of volumetric attacks—it holds for ICMP floods, SYN floods, UDP floods, and port scans **regardless of their specific implementation**.

The forward-to-backward packet ratio and forward-to-backward bytes ratio quantify this asymmetry directly. For legitimate bidirectional communication—a TCP file transfer, an HTTP session, a VoIP call—both ratios hover near 1.0 or at a modest imbalance reflecting the underlying protocol semantics. For a flood attack, the forward ratio approaches infinity as backward traffic goes to zero.

B. Why Not Use Connection State Tables

A stateful firewall approach to detecting asymmetry would maintain a connection state table and flag connections where responses are absent after a timeout. This approach is standard in enterprise firewalls but is unsuitable here for **two reasons**. **First**, maintaining per-connection state at 150,000 PPS generates memory pressure proportional to the number of concurrent half-open connections, which is exactly what a SYN flood maximizes. **Second**, the five-second window model is fundamentally stateless—state resets at each window boundary. The directionality ratios are computed within the window without any cross-window connection tracking, providing asymmetry information at constant memory cost regardless of flood intensity.

C. The Reply Rate as a Victim Responsiveness Signal

The `reply_rate` feature estimates what fraction of outgoing packets received a response within the same five-second window. A low reply rate from a destination that is normally responsive indicates either that the destination is overwhelmed and dropping packets or that the source is sending to closed ports or non-existent hosts—both of which are consistent with attack behavior. **For normal application traffic to a functioning server, the reply rate should be close to 1.0 for connection-oriented protocols. This feature provides a victim-side perspective that complements the source-side rate and entropy features.**

V. TCP Session Completeness Features

A. The Protocol Contract Violation Model

The TCP protocol specifies a well-defined lifecycle for every connection: a three-way handshake (SYN → SYN-ACK → ACK) establishes the connection, data transfer follows, and a four-way or three-way FIN sequence terminates it cleanly, with RST available for abnormal termination. Legitimate TCP applications overwhelmingly follow this contract. **Attack tools, by contrast, are specifically designed to violate it:** SYN flood tools send only the SYN packet and never complete the handshake; port scan tools probe ports and immediately move on, generating RST responses without establishing connections; TCP RST flood tools send only RST packets.

The SYN-to-ACK ratio, incomplete session ratio, and FIN count features operationalize this protocol contract as measurable quantities. **They require no knowledge of what application is running or what data is being transferred—they measure only whether the session lifecycle conforms to the TCP specification.**

B. Why These Flags Rather Than Others

TCP has six flag bits: URG, ACK, PSH, RST, SYN, FIN. The PSH flag indicates that the sending application has data to push immediately to the receiving application and **is common in legitimate interactive traffic (SSH, Telnet)**. The URG flag is rare in normal traffic and extremely rare in attack traffic as well, making it a weak discriminator in either direction. The existing schema already captures PSH counts for completeness. The features added in this group focus on SYN, ACK, FIN, and RST **because these four flags encode the session lifecycle state machine**—the full grammar of which connections are being opened, completed, and terminated—while PSH and URG encode application-layer urgency signals that are irrelevant to session completeness analysis.

C. The Incompleteness Ratio as a Zero-Day Indicator

Among the session completeness features, **the incomplete session ratio deserves particular attention as a potential zero-day attack indicator**. Any network behavior that generates large numbers of half-open sessions—SYN floods being the canonical example—will produce a high incompleteness ratio **regardless of what specific tool generated them**. **A novel attack tool not in the Snort signature database and not previously seen during training will still produce this feature value if it uses the same attack category**. The incompleteness ratio is therefore a **category-level indicator** rather than a tool-level indicator, which is precisely what is needed for generalization beyond the training distribution.

VI. Shannon Entropy Features

A. Information Theory as a Traffic Diversity Metric

Shannon entropy, borrowed from information theory, measures the diversity or unpredictability of a distribution. Given a set of observed values, entropy is maximized when all values are equally probable and minimized (approaching zero) when one value dominates. **Applied to network traffic, entropy quantifies how diversely a host is distributing its traffic across ports, destinations, or packet sizes**.

Flood attacks produce near-zero entropy in the dimension they are targeting: an ICMP flood generates all type-8 packets (entropy = 0 for ICMP type distribution), a UDP flood targets a single port (entropy = 0 for destination port distribution), and a port scan generates uniformly distributed destination port values (entropy = maximum). **These extreme values are a direct consequence of what the attack is trying to achieve—overwhelming a specific resource**—and not an artifact of the specific tool used.

B. Why Entropy Rather Than Raw Counts

The naive alternative to entropy is to count unique values: unique destination ports contacted, unique ICMP types seen. Counting unique values provides some discriminative power—a flood tool contacting one port will show `unique_dst_ports = 1`—**but it discards distributional information**. A host that contacts 100 ports, with 99% of packets going to port 80 and 1% distributed across 99 others, has `unique_dst_ports = 100` **but entropy near zero because the distribution is dominated by a single value**. Entropy captures this concentration in a single scalar, and its mathematical properties—specifically the fact that it is maximized

by the uniform distribution and minimized by the degenerate distribution—make it directly interpretable as a diversity measure.

C. The Multi-Dimensional Entropy Design

Four entropy features are computed over different dimensions of the traffic: destination port entropy (captures flood vs. scan distinction), payload size entropy (captures uniform-size attack tools vs. variable-size application traffic), source port entropy (captures random source port spoofing used in some flood tools), and ICMP type code entropy (captures ICMP flood vs. normal ICMP distinction). Computing entropy over multiple dimensions is necessary because no single dimension is universally discriminative. A UDP flood can use random source ports—appearing diverse in the source port dimension—while remaining maximally concentrated in the destination port dimension. Using only destination port entropy would miss the signal from a flood tool that spoofs its source port. The four-dimensional entropy profile provides a richer and more robust characterization.

D. Why Not Use Statistical Variance Instead

Variance is another measure of spread that could serve a similar role. For numeric features like packet sizes, both variance and entropy are reasonable choices. Entropy was preferred for port distributions because port numbers are categorical identifiers, not numeric quantities whose magnitude carries meaning. The difference between port 22 and port 23 is not numerically meaningful—they are simply different services. Treating port numbers as integers and computing their variance would produce a high value for any scan across a wide port range (ports 1–65535) but an artificially low value for a scan targeting only low-numbered ports (ports 1–100), even though both are equally concentrated attack behaviors relative to legitimate traffic patterns. Entropy treats port identifiers as categories and is therefore invariant to the numeric magnitude of the port numbers.

VII. ARP-Specific Behavioral Features

A. The Visibility Gap in Generic Flow Features

ARP spoofing is fundamentally different from the volumetric attacks addressed by the previous feature groups. It does not generate high packet rates, it does not create port diversity, and it does not violate TCP session lifecycle rules. A network under ARP spoofing attack may exhibit perfectly normal traffic rates, normal session completion patterns, and normal entropy across all port dimensions—because the attack operates at Layer 2 by poisoning the ARP cache, not by generating anomalous IP traffic. Generic flow features computed over IP packets are structurally incapable of detecting ARP spoofing because ARP operates below the IP layer and outside the scope of IP flow analysis.

This gap motivated the dedicated ARP feature group. Without it, ARP spoofing would be detectable only through Snort signature matching (Tier 1 of the detection engine), with no behavioral profiling fallback. For the deep learning model, this means ARP spoofing rows would have identical feature values to normal rows across all nine other feature groups, making them indistinguishable without explicit ARP-level signal.

B. The Unsolicited Reply Signal

The most discriminative single feature in this group is the unsolicited ARP reply count—the number of ARP replies sent by a host that were not preceded by an ARP request from the target host within the observation window. Legitimate ARP replies are responses to specific ARP requests; a host that generates replies without being asked is almost certainly poisoning the cache of other hosts. This asymmetry between ARP requests and replies has no benign explanation at significant volumes. A small number of gratuitous ARPs (where a host announces its own IP-to-MAC mapping) is normal during device startup, but a sustained stream of unsolicited replies targeting another host's IP-to-MAC binding is the defining signature of an active spoofing attack.

C. The Persistent Binding Table Requirement

MAC-to-IP binding change tracking requires state that persists across five-second windows—a legitimate device rarely claims different IP addresses, so a binding change detected today is still meaningful tomorrow. This is handled through a module-level dictionary (`_mac_to_ip_history`) that accumulates all IP addresses a given MAC has claimed since the controller started. The magnitude of this dictionary grows at most at the rate of network events (new devices joining, DHCP renewals) and is bounded in practice by the number of devices in the topology.

The alternative—tracking binding changes only within each five-second window—would fail to detect slow ARP poisoning where the attacker gradually shifts the binding over many windows, keeping the per-window change count below any reasonable threshold. The persistent binding table approach is immune to this evasion because the historical record grows monotonically; any deviation from a device's established binding pattern is visible regardless of how slowly it occurs.

VIII. Packet Size Distribution Features

A. The Uniformity Signature of Attack Tools

Commercial and open-source flood tools generate packets of fixed or configurable size. hping3, for example, generates ICMP packets of 28 bytes by default (the minimum ICMP echo request), with the payload size controllable via the `-d` flag. The packets generated in a single flood session are all identical in size. The standard deviation of packet sizes in an attack flow is therefore near zero—often exactly zero if the tool does not randomize sizes.

Legitimate application traffic, by contrast, is highly variable in packet size because different application events (small acknowledgments, large data segments, retransmissions) produce packets of different sizes. A file transfer using TCP generates packets ranging from 40-byte acknowledgments to 1460-byte maximum segment size data packets. The standard deviation of packet sizes in a legitimate flow is correspondingly large.

B. Why Standard Deviation Rather Than Min-Max Range

The range ($\text{max_size} - \text{min_size}$) might seem sufficient to capture size variability—a flow with only 28-byte packets has $\text{range} = 0$, while a flow with mixed-size packets has a large range. However, range is highly sensitive to outliers and does not capture the distribution of sizes across all packets. **A flood tool that occasionally sends one large packet to probe a maximum transmission unit will have a non-zero range even if 99.9% of its packets are identical in size. Standard deviation weighs all packets equally and is resistant to this manipulation.** The packet size variance feature is derived directly from the standard deviation and serves primarily as an additional input for the deep learning model, which may extract different patterns from the squared deviation.

C. The Small and Large Packet Ratio Features

The small packet ratio (fraction of packets under 100 bytes) and large packet ratio (fraction of packets over 1000 bytes) complement the standard deviation by providing interpretable boundary statistics. **SYN flood packets are always small**—the SYN packet has no data payload and typically spans 60–64 bytes. UDP amplification attack responses are always large—the amplification factor depends on the response being substantially larger than the request. These ratios directly encode the known physical characteristics of specific attack categories in a form that a neural network can associate with labels during training.

IX. Port Behavior Features

A. Distinguishing Floods from Scans at the Port Level

Flood attacks and port scans occupy **opposite extremes of the destination port diversity spectrum**. A flood attack typically directs all traffic to a single destination port—the service being targeted—**producing a destination port standard deviation of zero and a top port ratio of 1.0**. A port scan contacts as many distinct ports as possible—the goal is to enumerate which ports are open—**producing a high destination port standard deviation and a top port ratio near zero**.

The existing `unique_dst_ports` count captures the number of distinct ports but not their distributional properties. Two flows can have the same `unique_dst_ports` count but completely different concentrations—one dominated by a single port and one uniformly distributed across all of them. The standard deviation of destination port numbers, the top port ratio, and the sequential port score **together provide a complete characterization of both the concentration and the ordering pattern of port usage**.

B. The Sequential Port Score and Its Specificity

The sequential port score—the fraction of consecutive port pairs in the observed sequence where the second port is within two of the first—is **specifically designed to detect the default scanning behavior of nmap**, which probes ports in sequential order unless randomization is explicitly requested. This feature is not intended to be a universal attack discriminator; **it is a targeted feature for a specific tool behavior that produces a specific, recognizable pattern**.

The justification for including tool-specific features alongside general behavioral features is the following: while the deep learning model should generalize beyond specific tools, its training data is necessarily generated by specific tools. Including features that are highly discriminative for the tools used during

training—alongside features that capture general behavioral principles—**gives the model both a strong training signal and the general features needed for generalization**. A model trained with only general features **may fail to converge on the correct decision boundaries due to insufficient signal**; a model trained with **only tool-specific features will overfit**.

C. Well-Known and Ephemeral Port Ratios

Well-known ports (0–1023) host standard services—HTTP on 80, SSH on 22, DNS on 53, SMTP on 25. Ephemeral ports (49152–65535) are dynamically assigned by the operating system to outgoing connections. A host that directs nearly all of its traffic to well-known ports is **likely engaging in service-targeted activity—which could be legitimate (a web client) or malicious (a targeted flood or service exploitation attempt)**. A host that directs traffic primarily to ephemeral ports is engaging in peer-to-peer or application-specific communication. **These ratios provide context for the port entropy and standard deviation features without adding computational complexity**.

X. Broadcast and Multicast Environment Features

A. The Mininet-Specific False Positive Problem

The broadcast and multicast features address a challenge **unique to the Mininet emulation environment rather than a general attack detection challenge**. During a pingall operation in Mininet—a diagnostic command that sends ICMP echo requests between all host pairs—the software switch generates broadcast ARP storms as it learns the network topology. These storms can produce several thousand packets within a single five-second window from a single source, with near-zero inter-arrival coefficient of variation (the broadcasts arrive in rapid succession), near-unity top destination ratio (broadcast address), and high ICMP rate.

In the absence of broadcast-specific features, these legitimate operational events are indistinguishable from a real ICMP flood at the feature level. The existing detection system handles this through elevated thresholds (15,000 packets per window rather than 1,000), **but elevated thresholds reduce sensitivity for genuine attacks**. The broadcast ratio, multicast ratio, and ARP broadcast ratio features give the deep learning model a direct signal for classifying broadcast-storm windows as normal rather than requiring the model to infer this from the elevated threshold alone.

B. Why These Features Are Environment-Specific but Necessary

A critic might argue that environment-specific features reduce the generalizability of the dataset and the model trained on it. This argument would be valid if the goal were to produce a general-purpose IDS dataset comparable to CIC-IDS2017 or UNSW-NB15. **The goal here is different: to train a model that operates specifically within this SDN topology, with this controller architecture, on this emulated network**. A model that achieves 99% accuracy in the lab **but generates constant false alarms during routine pingall operations is operationally worthless**. The broadcast features are necessary for building a deployable system within this environment.

Furthermore, real networks also generate broadcast traffic during normal operation—ARP resolution, DHCP discovery, Spanning Tree Protocol convergence. **A model trained to recognize broadcast context as a normal signal will generalize correctly to physical deployments**; a model trained without this context will generate false alarms whenever the physical network undergoes routine broadcast activity.

XI. Flow-Level Context and Device History Features

A. The Baseline Maturity Signal

The Z-score deviation features in the device behavioral profile (`device_pkt_rate_deviation`, `device_byte_rate_deviation`, `device_payload_size_deviation`) **compare each window's traffic against the device's historical average**. These features are powerful for detecting behavioral changes but have a well-known cold-start problem: during the first 180 seconds of a device's lifetime in the network, the historical average is based on too few observations to be statistically reliable. **A device that begins sending legitimate high-volume traffic within its first 60 seconds of operation will appear anomalous relative to its very short history.**

The `is_baseline_mature` binary flag encodes exactly when the historical baseline becomes reliable—after 20 observed flows and 180 seconds. Including this flag in the feature set allows the deep learning model to learn a conditional behavior: apply Z-score deviation features with full weight when `is_baseline_mature` = 1, and treat them with skepticism when `is_baseline_mature` = 0. Without this flag, the model must infer this conditional from `device_total_flows` and `device_age_seconds`, **which requires the model to learn the threshold values (20 flows, 180 seconds) from data rather than having them directly encoded.**

B. The Flows Per Window Feature and Connection Flooding

The `flows_per_window` feature counts how many distinct flow entries a source IP generated within a single five-second window. **This feature is specifically designed to detect connection flooding**—an attack where the goal is not to generate high packet volume to a single destination but to open as many simultaneous connections as possible, **exhausting the target's connection table. Connection flooding attacks can appear benign in all rate-based and entropy features** because each individual flow has low packet count and unremarkable timing characteristics. **Only the aggregate number of flows from a single source reveals the anomaly.**

This feature also distinguishes network scanning from port scanning: a port scan generates high `unique_dst_ports` from a single source to a single destination, while a network scan generates high `flows_per_window` because each probed IP address constitutes a separate flow entry. The `flows_per_window` **feature captures the horizontal scan pattern (one source, many destinations) that `unique_dst_ports` misses.**

XII. Metadata Columns and Their Role in Dataset Integrity

A. The Distinction Between Features and Metadata

The metadata columns (`meta_attack_tool`, `meta_attack_intensity`, `meta_mininet_event`, `meta_session_id`, and others) are explicitly excluded from the training feature set. They serve a different function: they provide ground-truth annotations about the experimental conditions under which each row was generated. This distinction is critical because the metadata columns would constitute data leakage if included as model inputs—a model trained with `meta_attack_tool` as a feature would learn to predict labels directly from the tool identifier rather than from the behavioral features, producing artificially high training accuracy and zero generalization to real deployments.

B. The Reproducibility Function

The metadata columns serve two reproducibility functions. First, they enable post-hoc validation of the dataset: by filtering on `meta_attack_tool = hping3` and verifying that all such rows have non-zero label values, one can confirm that the `ATTACK_START/ATTACK_STOP` metadata signaling was working correctly during data collection. Second, they enable ablation studies: by filtering on `meta_mininet_event = pingall` and examining the feature distributions, one can verify that the broadcast-related features correctly distinguish pingall windows from genuine flood windows.

C. The Controller Load Features

The `meta_controller_load` column records the processing latency per flush—the time required to compute all features for one five-second window. This metadata is valuable for understanding the computational limits of the system under attack conditions: a sustained 150,000 PPS flood may push the feature computation time well above the five-second window duration, causing windows to back up. If the `meta_controller_load` values show window computation times exceeding 5,000 milliseconds during flood attacks, the feature set should be profiled and optimized before deployment.

XIII. Comparison Against Benchmark Datasets

Table I compares the feature coverage of the enhanced schema against two widely cited benchmark network intrusion detection datasets.

TABLE I
Feature Coverage Comparison Across IDS Benchmark Datasets

Feature Group	This Work	CIC-IDS2017	UNSW-NB15
Flow-level mechanics	✓ 21 features	✓ 18 features	✓ 15 features
Timing & inter-arrival rhythm	✓ 7 features	✓ Partial (IAT mean/std)	✗ Not present
Directionality & asymmetry	✓ 7 features	✓ Fwd/Bwd split	✓ Partial
TCP session completeness	✓ 4 features	✓ Flag counts only	✓ Flag counts only
Shannon entropy (multi-dim)	✓ 4 features	✗ Not present	✓ 2 features

Feature Group	This Work	CIC-IDS2017	UNSW-NB15
ARP-specific behavioral	✓ 7 features	✗ Not present	✗ Not present
Packet size distribution	✓ 4 features	✓ Partial	✓ Partial
Port behavior patterns	✓ 6 features	✓ Partial	✓ Partial
Broadcast/multicast context	✓ 4 features	✗ Not present	✗ Not present
Device behavioral history	✓ 16 features	✗ Not present	✗ Not present
Total feature columns	~85+	78	49

The comparison in Table I reveals two feature groups that are unique to this work and absent from both benchmark datasets: the ARP-specific behavioral features and the device behavioral history features. The ARP group is absent from benchmarks because most benchmark datasets are generated on physical networks where ARP spoofing has a distinct network-layer signature, or because the benchmarks focus on network-level rather than host-level attack scenarios. The device behavioral history group is absent because benchmark datasets are typically static snapshots rather than live systems with persistent per-device profiling.

The device behavioral history features are particularly significant for zero-day detection. CIC-IDS2017 and UNSW-NB15 train models on absolute feature values—the model learns that a specific PPS value is malicious. The device deviation features in this work train the model on relative deviations from a device's own historical baseline—the model learns that a significant departure from normal behavior is malicious, regardless of the absolute value. This distinction is the mathematical foundation for the claim that the system can detect behavioral anomalies consistent with previously unseen attack patterns.

XIV. Conclusion

The feature set documented in this work was not selected by enumeration—by listing all plausible packet header fields and including them all—but by reasoning backward from the attack mechanisms under consideration to the measurements that expose those mechanisms most directly. Each feature group was motivated by a specific hypothesis about how attacks differ from normal traffic, each hypothesis was operationalized through a specific mathematical or statistical measure, and each alternative measure was evaluated and rejected based on its computational cost, sensitivity to the Mininet amplification artifact, dependence on attack-specific tool behavior, or failure to generalize across attack intensity levels.

The result is a feature set that operates at three levels of abstraction simultaneously. At the flow level, it captures the mechanics of individual connections—their duration, volume, timing regularity, and protocol conformance. At the device level, it captures behavioral history—how much a device's current behavior deviates from its own established baseline. At the network level, it captures global state—the entropy of source IP distributions, the total active flow count, the prevalence of broadcast traffic. This multi-scale approach is necessary because different attack types manifest their anomalies at different scales: a SYN

flood is detectable at the flow level from session incompleteness, a slow port scan is detectable at the device level from new destination ratios, and a distributed denial-of-service attack is detectable at the network level from the entropy of source IP addresses.

The deep learning model trained on this dataset inherits the multi-scale structure of the features. Convolutional layers operating on flow-level features will learn local feature interactions—the relationship between `syn_count` and `ack_count` that signals a SYN flood. Recurrent or attention-based layers operating on temporal sequences of windows from the same device will learn the temporal pattern of behavioral change that precedes and follows an attack. The combination provides detection capability at both the within-window and cross-window levels, which is the foundation for the system's intended zero-day detection capability.